VILLAGE POND PLANNING SYSTEM · PHASE 2

Pond Catchment Analysis

Upload a contour map as KML or KMZ. The service returns a village pond site, the ground that drains into it, the storage the pond holds, and the water that ground yields in a year.

Akshat 1 September 2026 FastAPI · numpy · scipy Live on stu68_sys1

01 Links

GITHUB REPOSITORY **Akshats-git / Pond-Catchment-Analysis**	MAIN API ROUTE **POST /api/v1/analyzeContour**
INTERACTIVE API DOCS **10.1.75.53:5229/docs**	DEMO PAGE **10.1.75.53:5229**
HEALTH CHECK **10.1.75.53:5229/health**	WRITTEN API REFERENCE **docs/API.md**
METHOD AND VALIDATION **docs/METHODOLOGY.md**	SAMPLE CONTOUR MAP **data/contours_1m.kml**

One command tests the whole thing.

```
curl -F contour_map=@data/contours_1m.kml http://10.1.75.53:5229/api/v1/analyzeContour
```

Or open the demo page in a browser and drop the sample file on it.

02 What was built

A FastAPI service in Python. One endpoint does the analysis. Four more exist so a client can draw the input, get a picture, look up rainfall and check that the service is up.

METHOD	PATH	WHAT IT DOES
POST	/api/v1/analyzeContour	Contour map in. Site, catchment, storage and yield out
POST	/api/v1/findCatchment	Same endpoint under the name the brief uses

METHOD	PATH	WHAT IT DOES
POST	/api/v1/renderMap	The same analysis drawn as a PNG map
POST	/api/v1/contours	The uploaded map back as drawable lines, with no analysis
GET	/api/v1/rainfall	Ten years of daily rainfall for one point
GET	/health	Liveness

The stack is Python 3.12, FastAPI, numpy and scipy. There is no GDAL, no pysheds and no rasterio. The terrain engine is about 900 lines of numpy. That is what lets the service install and run inside a 512 MB container with no system packages.

Nothing in the answer is hard coded to the sample sheet. Grid size, smoothing width and stream threshold are all derived from the uploaded file. No coordinate or elevation literal exists outside `data/` and `tests/`.

---

03 How the catchment is estimated

Six steps. Each one can be checked on its own.

1. **Contour lines to height points.** Every vertex of every line is a known point with a longitude, a latitude and a height. The sample gives 159,113 points across 1,355 lines. Heights are read from the `z` coordinate first, then from `ExtendedData`, then from the placemark name. A file that carries its heights in any of those three places is read without being edited first.

2. **Points to a grid.** Coordinates are projected to local metres. Delaunay triangulation and linear interpolation put the heights on a square grid. Cell size is the mean contour spacing divided by four, so the grid follows the detail the map carries.

3. **Smoothing.** Raw interpolation between contour lines leaves flat stair steps instead of a hillside, and water runs along a step instead of down it. A NaN aware Gaussian removes them. This step is worth up to 12.79% of the catchment area on a valley whose answer can be worked out on paper. It is the least obvious step in the pipeline and the one that matters most.

4. **Fill the pits.** Water should not sit in a hole that the interpolation invented.

5. **Route the water.** D8 flow direction. Each cell drains to its steepest of eight neighbours. Accumulation then counts how many cells drain through every cell. The channel network is a result of this step. It is never drawn or digitised.

6. **Trace the catchment.** Walk the flow arrows backwards from the outlet. The catchment is every cell whose water ends up there.

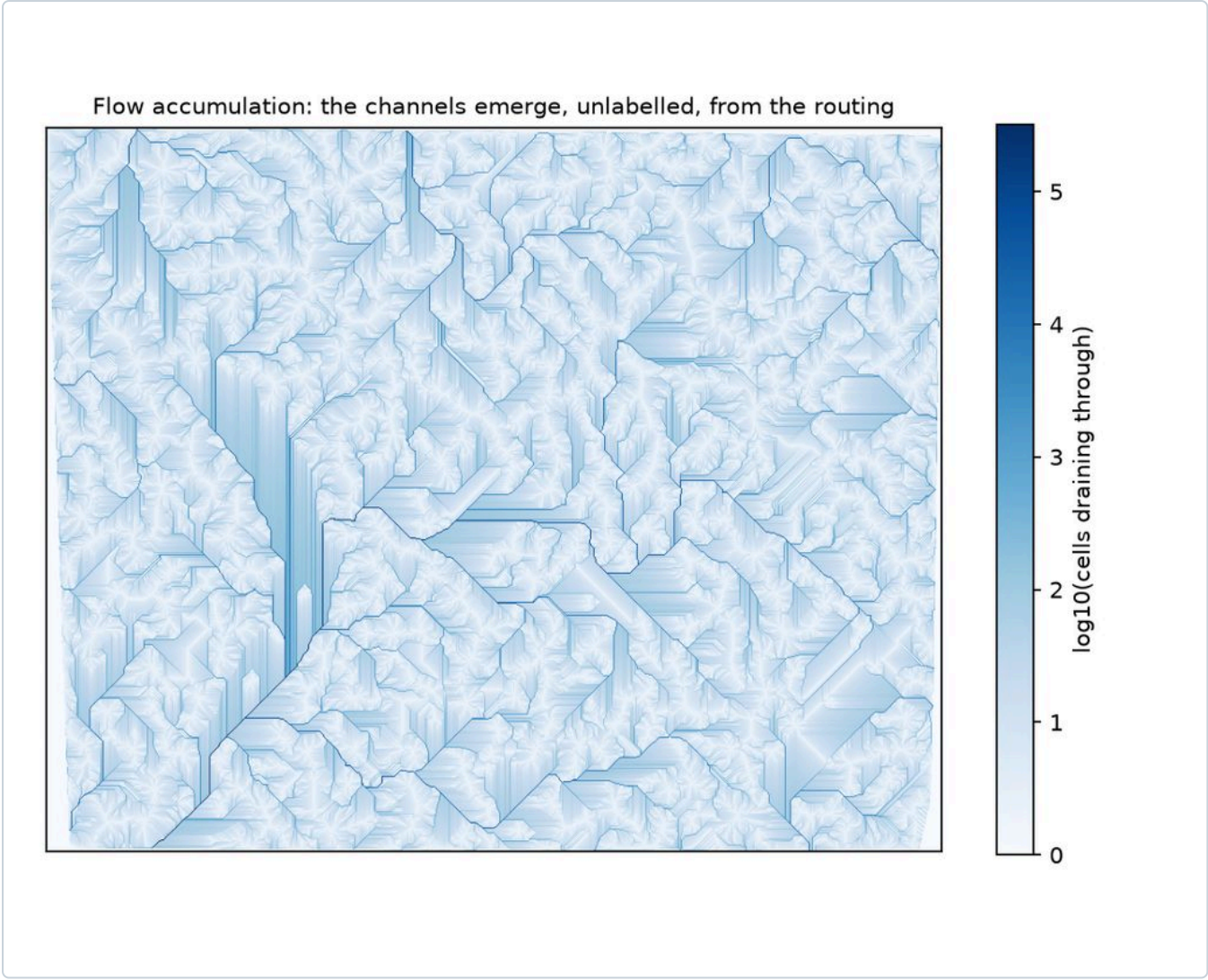

Figure 1. Flow accumulation on the sample sheet, log scale. Every channel here came out of step 5. Nothing in this image was digitised from the map.

Where the pond goes

The site search starts from that channel network. Find the streams, keep ground flat enough to build on, rank by how much land drains into each spot, then remove each pick's whole catchment so the alternatives are separate basins.

> THE RULE THAT MATTERS
>
> Ranking by catchment area alone asks for the cell that the most water passes through. On a sheet with a river across it, that cell is the river. The first working version put its top site in the middle of the Shivnath. A 3 m structure there is a dam, not a village pond.

So any channel already draining more than 150 ha counts as a watercourse, and a site has to stand at least 3 m above the watercourse it drains into. The threshold is in absolute hectares, never a share of the sheet, or the biggest channel on a 20 ha farm map would be called a river.

Checked against the OpenStreetMap water layer, which knows nothing about this terrain model, the rule takes candidate ground standing in water down to zero.

CANDIDATE FILTER	CANDIDATE CELLS	STANDING IN OSM WATER
Stream and buildable slope only	2,413	310 (12.8%)
Plus clear of the watercourse	566	0 (0.0%)

04 Demonstration on the provided contour map

`data/contours_1m.kml` sent to the live service with default settings. This is the run that produced every number below.

```
curl -F contour_map=@data/contours_1m.kml http://10.1.75.53:5229/api/v1/analyzeContour
```

What was read. 1,355 contour lines, 159,113 vertices, 32 levels at 1.0 m spacing, 267 m to 298 m, covering 830.8 ha. One line had no readable height and was skipped. The response says so instead of hiding it.

The grid. 719 by 888 cells at 3.65 m, from a mean contour spacing of 14.62 m. Smoothing sigma 1.83 m, which moved no point by more than 1.155 m. 2.5% of the frame is no data because it falls outside the mapped area.

The search. Stream threshold 4.2 ha giving 6,536 stream cells. 160,890 cells flat enough to build on. 230,082 cells clear of the watercourse. 566 candidates survived every rule.

Recommended site, 21.243922 N 81.289998 E

CATCHMENT	STORAGE AT 3 M	ANNUAL RUNOFF	FILL RATIO	RESPONSE TIME
66.3 ha	**20,993** m³	**127,040** m³	**6.05** x	**13.8** s

Edge contact	3.0%, so the catchment sits inside the sheet and the area is a real measurement
Relief above the outlet	18.0 m
Longest flow path	2,270 m, time of concentration 49 minutes
Outlet slope	1.8%, against a 3% limit
Height above the nearest watercourse	4.0 m, against a 3 m rule
Water spread at 3 m	12,699 m^2

The service also says why it picked the spot.

- largest upstream area: 66.3 ha, 8% of the mapped sheet
- buildable slope at the outlet (1.8%, limit 3%)
- 18 m of relief above the outlet
- sits 1.2 m below the surrounding ground
- stands 4.0 m above the nearest watercourse, so the pond is clear of the channel

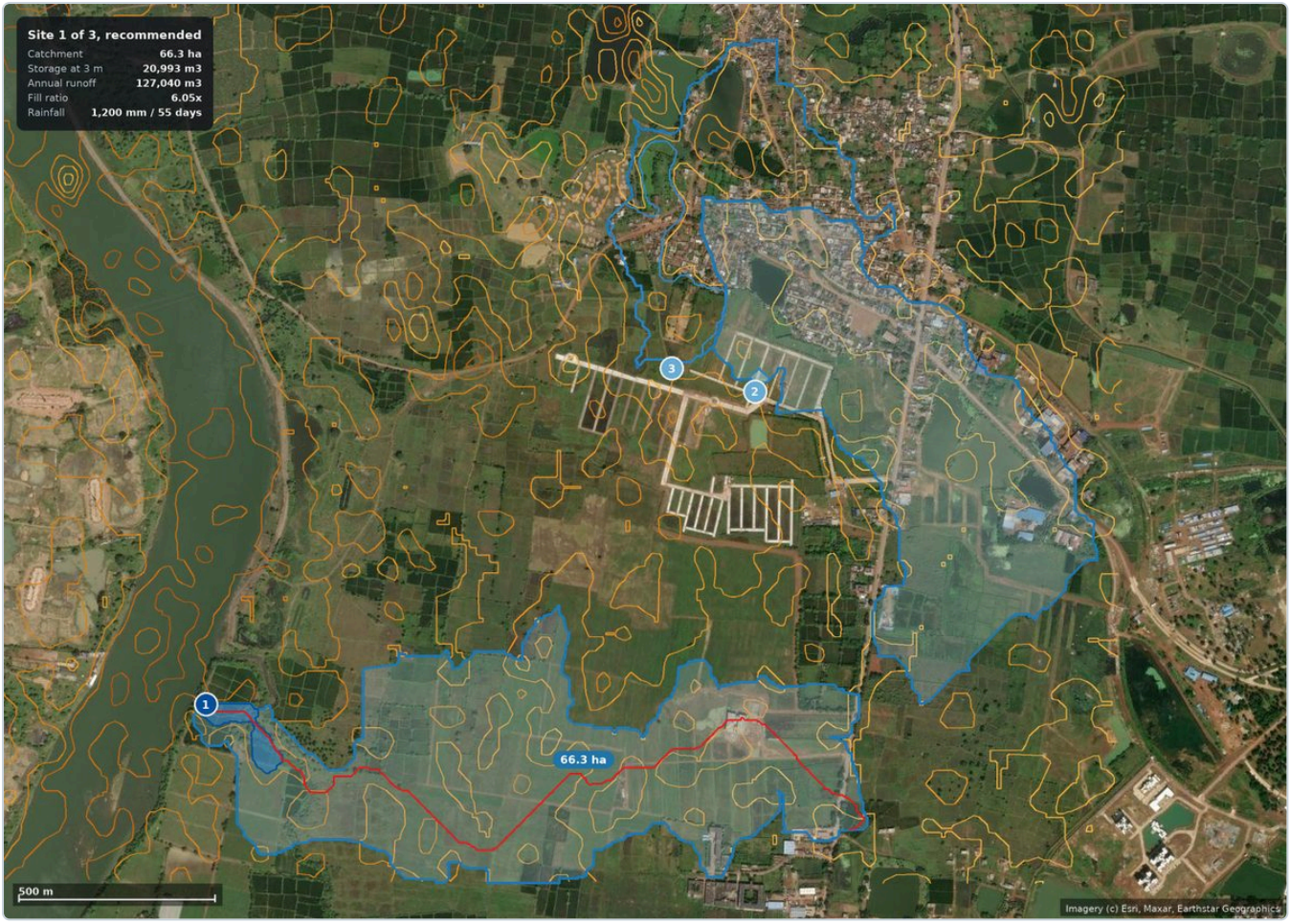

Figure 2. The answer as `/api/v1/renderMap` draws it, straight from the live service. Site 1 is the recommended pond. The blue outline is its 66.3 ha catchment, the red line is the longest flow path, and the orange lines are the uploaded contours. The boundary runs along the ridges, which is the check that no number can make for you.

Alternatives

RANK	LOCATION	CATCHMENT	STORAGE AT 3 M	ANNUAL RUNOFF
1	21.243922, 81.289998	66.3 ha	20,993 m^3	127,040 m^3
2	21.251095, 81.303488	56.5 ha	682 m^3	108,213 m^3
3	21.251624, 81.301445	24.9 ha	146,505 m^3	47,614 m^3

These are three separate basins, not three points on one stream. The spread in storage is the terrain talking. Site 2 is a channel position that would have to be dug out. Site 3 sits in a natural hollow that holds water almost for free.

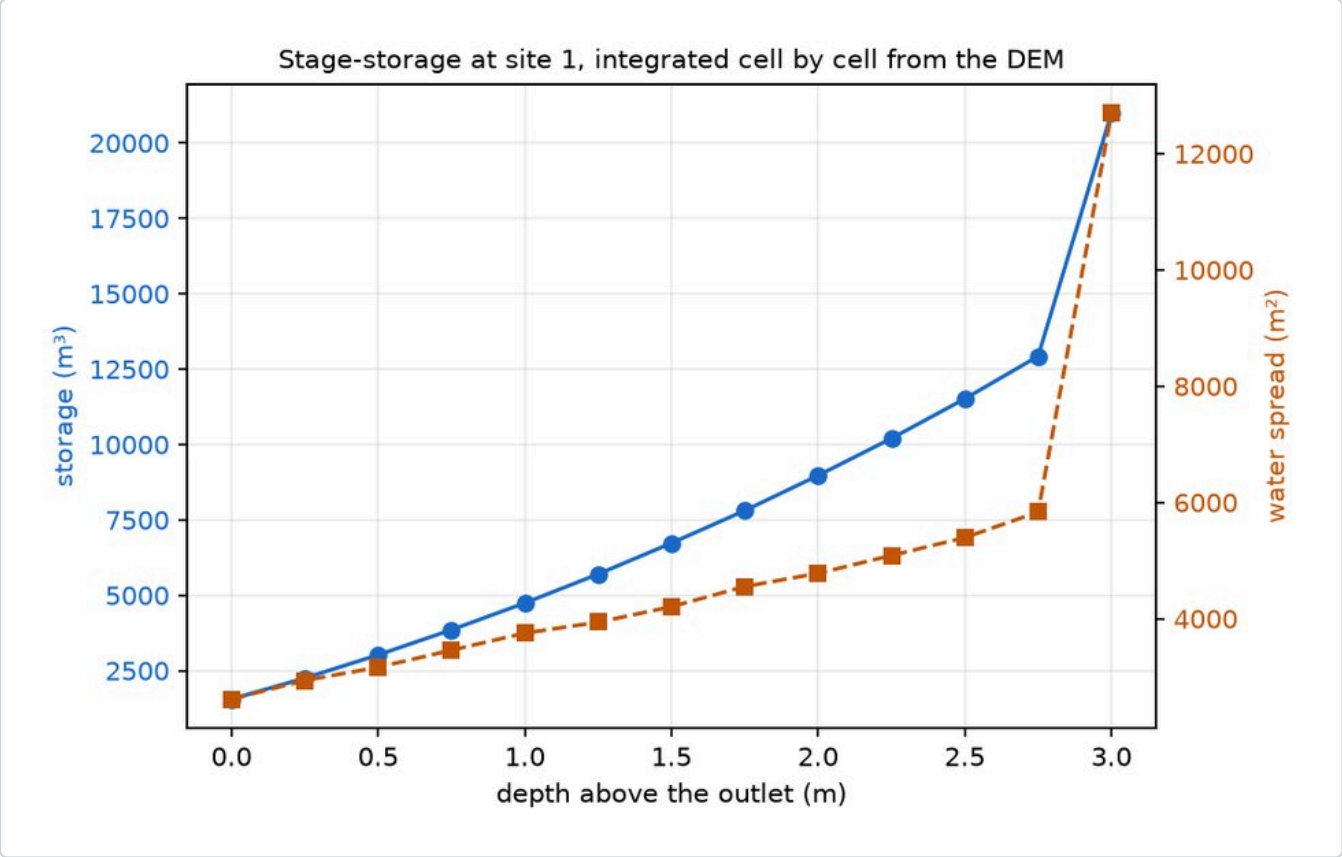

Figure 3. Stage and storage at site 1, integrated cell by cell off the grid rather than assumed from a shape. A frustum estimate of the same pond gives 21,081 m^3 against the integral's 20,993 m^3. They agree to 0.4%.

Water

Rainfall for this run was 1,200 mm over 55 rain days. The service asks Open-Meteo for ten years of daily records at the site. Open-Meteo answers HTTP 429 from the lab's shared IP, so the run fell back to a documented Raipur climatology and said so in `warnings`. An analysis is never blocked by the weather. You can also send your own figure with `rainfall_mm` and `rain_days`.

SCS-CN is an event model. Run a whole year of rain through it as one storm and it reports a runoff coefficient of 0.92, which no catchment on earth produces. Run it per rain day and add the days up and it gives 0.16, which is 191.5 mm of runoff from 1,200 mm of rain across 20 contributing days. That is a factor of 5.77 between the careless answer and the defensible one. Both are in the response, as `runoff_coefficient` and `single_event_coefficient`, because the wrong number is what shows why the right one is right.

At a fill ratio of 6.05, far more water arrives than a 3 m pond here can hold. The pond fills early in the monsoon and then spills. The honest conclusion for this site is that the spillway matters more than the capacity.

05 API documentation

Interactive documentation is live at 10.1.75.53:5229/docs. FastAPI generates it from the same models that serialise the responses, so it cannot drift from what the endpoint returns. The full written reference is docs/API.md.

POST /api/v1/analyzeContour

`multipart/form-data`. Only the file is required. Everything else has a derived default.

FIELD	TYPE	DEFAULT	MEANING
`contour_map`	file	required	The contour map, .kml or .kmz. Max 64 MB
`grid_resolution`	float	derived	Grid cell size in metres, 2 to 20
`top_n`	int	3	How many separate basins to return
`lat, lon`	float	none	A pour point you choose yourself. Sending both skips the site search
`curve_number`	float	75	SCS curve number, 30 to 98. How readily the ground sheds rain
`rainfall_mm`	float	live	Yearly rainfall. Left out, the service looks it up
`rain_days`	int	live	Days a year that rain falls on
`target_depth_m`	float	3.0	How deep the pond is to be built
`ensemble`	bool	false here	Re-trace each site on three more grids for an error bar

The file field is `contour_map`. The older name `file` still works so that a client written before the name was fixed does not break. In Postman, use POST, body type form-data, one row of type File named `contour_map`.

The 200 response carries `input`, `dem`, `parameters`, `recommended_site`, `alternative_sites`, `search`, `geojson`, `warnings` and `timing_ms`. The `geojson` block is a complete FeatureCollection with a catchment polygon, a pond footprint and an outlet per site, plus the longest flow path for the recommended one. It pastes straight into geojson.io.

The other endpoints

- **POST /api/v1/renderMap** takes the same fields and returns a PNG instead of JSON, plus `width`, `height`, `basemap`, `contours`, `frame` and `legend`. A picture cannot carry a caveat, so every warning the JSON would have returned comes back in the `X-Pond-Warnings` header. Read it.
- **POST /api/v1/contours** hands the uploaded lines straight back as a styled FeatureCollection without analysing anything. It runs the same parser, so the lines it draws are the lines the analysis read, and it answers in about two seconds instead of fifteen. This is what lets a reader check a catchment boundary against the ground it claims.
- **GET /api/v1/rainfall** exposes the rainfall feed on its own, so a page can show the figure before committing to a full analysis.
- **GET /health** does no work on purpose, so it answers before the first analysis rather than after one. It also reports whether this host can run the ensemble.

Errors

Every error from every endpoint has one shape.

```
{
  "status": "error",
  "code": "no_buildable_ground",
  "detail": "What went wrong, in a sentence.",
  "hint": "What to change about the request."
}
```

`code` is stable and meant for machines. `detail` and `hint` are for a person. The status code separates three cases. **400** means the file cannot produce an answer. **413** means more data than the service will take on. **422** means the request was understood but cannot be honoured as asked. A timeout past 120 seconds returns **504**. The full code list is in docs/API.md.

Warnings are not errors. A 200 response can carry warnings and the sample carries four: the skipped contour line, the rainfall fallback, the fact that the climatology is not an observation, and the fact that a 1 m contour interval bounds how precise any depth can be. They are part of the answer, so show them.

Commands you can run right now

Health

```
curl http://10.1.75.53:5229/health
```

The full analysis

```
curl -F contour_map=@data/contours_1m.kml \
     http://10.1.75.53:5229/api/v1/analyzeContour
```

Your own rain figures, a coarser grid, five sites

```
curl -F contour_map=@data/contours_1m.kml -F rainfall_mm=1200 -F rain_days=55 \
     -F grid_resolution=5 -F top_n=5 \
     http://10.1.75.53:5229/api/v1/analyzeContour
```

A pour point you picked yourself

```
curl -F contour_map=@data/contours_1m.kml -F lat=21.243922 -F lon=81.289998 \
     http://10.1.75.53:5229/api/v1/analyzeContour
```

The answer as a picture

```
curl -X POST -F contour_map=@data/contours_1m.kml -F frame=sites \
     http://10.1.75.53:5229/api/v1/renderMap -o catchment.png
```

Just the contour lines, no analysis

```
curl -X POST -F contour_map=@data/contours_1m.kml \
     http://10.1.75.53:5229/api/v1/contours -o contours.geojson
```

Rainfall on its own

```
curl "http://10.1.75.53:5229/api/v1/rainfall?lat=21.2439&lon=81.2900"
```

---

06 Evaluation

The method is checked against things that can be known without it.

A valley with a provable answer

The surface `z = 0.05·|x| + 0.01·y` over 1 km by 1 km. Steepest descent is pure minus x, so the catchment of a channel point at `y = Y` is provably `1000 × (1000 − Y)` m². That surface is written out as a contour KML and read back through the same pipeline the real map uses.

GRID	Y = 250	Y = 500	Y = 750
Smoothing off	−4.30%	0.00%	−12.79%
Sigma 2.5 m, 5 m grid	**0.00%**	**0.00%**	**0.00%**
Sigma 2.5 m, 10 m grid	**0.00%**	**0.00%**	**0.00%**

The 12.79% has a traceable cause. Seven grid rows above the outlet contributed one cell each instead of 200, because hillslope water ran along a flat stair step and joined the stream below the outlet. Smoothing fixes it exactly. That is the evidence for step 3.

Mass balance on the real map

Every cell drains to exactly one outlet, so the basin areas have to add up to the mapped area.

```
sum of all basin areas = 8.309123 km2
mapped area            = 8.309123 km2      difference: 0.00000000%
```

No cell is lost, double counted or routed into a loop. The check is done two ways that share no code: partitioning downstream, and delineating every outlet upstream.

The sub-tile test

The sample sheet is clipped to each of its four quadrants and analysed again from scratch. Each quadrant works out its own grid size and finds its own largest catchment, and the four answers differ by more than a factor of two. A result that came from a hard coded coordinate or a cached site would pass every other test and fail this one.

Structural variants

Eleven synthetic KML and KMZ files are carried all the way to a delineated catchment: 3D coordinates, ExtendedData, folder names, polygon rings, MultiGeometry, no XML namespace, spaced coordinates, a duplicate labels folder, a stray 3D polygon and a KMZ archive. Parsing a file is not the same as being able to analyse it.

Test suite

```
$ pytest
451 passed in 135.13s (0:02:15)
```

No test needs the network. `tests/conftest.py` switches the live rainfall fetch off, and the Open-Meteo provider is tested against a payload built inside the test.

Live endpoint checks

Run from outside the container on 1 September 2026.

CHECK	RESULT
`GET /health`	200
`POST /analyzeContour` with the sample map	200, 41.7 KB, 13.8 s
`POST /findCatchment`, either field name	200, identical answer
`POST /renderMap`, `frame=sites`	200, PNG, warnings in the header
`POST /contours`	200, 927 KB, 2.1 s
`GET /api/v1/rainfall`	200, fell back to climatology and said so
`GET /` and `GET /docs`	200
`POST` with no file	422 missing_file, naming the field and showing the curl flag
`POST` with `ensemble=true`	422 ensemble_unavailable, naming the memory limit

---

07 Deployment

The service runs on the lab container `stu68_sys1` and is reachable at http://10.1.75.53:5229.

```
laptop  ->  10.1.75.53:5229  ->  container 172.17.0.30:5000  ->  uvicorn on 0.0.0.0:5000
```

`run.sh` sets the environment and restarts uvicorn if it dies. It takes a lock file, so starting it twice is harmless. To bring the service back after a container restart:

```
ssh -p 2229 student@10.1.75.53
setsid ~/PondCatchmentAnalysis/run.sh >/dev/null 2>&1 </dev/null &
```

The one constraint worth reporting

The container is capped at 512 MB. A single analysis peaks near 300 MB and fits. The four grid ensemble peaks at 581 MB and does not. Two changes closed that gap, and both came from testing the deployed service rather than from reading the code.

Two analyses at once used to kill the service. The second request did not queue. Both peaked together, the kernel killed the worker they shared, and both clients got an empty reply. Analyses now run one at a time and the rest queue. Three uploads at once return 200 in 15 s, 27 s and 39 s with no failure.

An explicit `ensemble=true` used to kill it too. A reader of `/docs` sees `ensemble` as a documented parameter and will try it. That request now returns 422 `ensemble_unavailable` and names the limit instead of taking the worker down.

So the ensemble is off on this deployment and responses from the container carry no cross-resolution error bar. Everything else is unchanged. This is a property of the container, not of the code. Set `POND_API_ALLOW_ENSEMBLE=true` on a host with more memory and the error bar comes back.

Every tunable lives in `app/config.py` with its reason written next to it, and every one is overridable from the environment with a `POND_` prefix. That is how the ensemble was switched off here without changing a line of code.

---

08 Extending to Phase 3

The layering exists so that Phase 3 changes one file at a time. `app/core/` holds the analysis. `app/providers/` holds the swappable data sources. `app/routers/` is the HTTP surface and nothing else. `app/pipeline.py` is the only place they are wired together.

PHASE 3 REQUIREMENT	THE SEAM THAT IS ALREADY THERE
DEM from an elevation API instead of KML	`pipeline.py` builds a DEM object and everything downstream takes that object, never the KML. A provider that returns a DEM replaces `dem_builder` alone
Real rainfall	`providers/rainfall.py`, already swapped once from a constant to Open-Meteo behind an unchanged interface
Larger regions or a different CRS	The Projection interface. Swap local ENU for UTM
Faster or GPU flow routing	The TerrainEngine interface. pysheds drops in behind it
Storing results	`pipeline.py` returns a plain dataclass, so it serialises without work
Land availability from the OpenCV service	`select_pond_sites(..., exclusion_mask=)`. Ground to leave alone is masked out of the candidate pool, and excluding everything returns its own error code instead of an empty answer

The rainfall seam is the one that has been proven rather than promised. This service first shipped with a documented regional constant. The live Open-Meteo feed replaced it inside this phase and nothing outside `providers/rainfall.py` had to change. The constant is still there as the fallback, so the interface is carrying two implementations at once.

---

09 Limits worth knowing

- **Terrain is not tenure.** The model says where water collects. It cannot say whether that spot is a house, a road or a field somebody owns. That is what `exclusion_mask` and Phase 3's land availability service are for.

- **Rainfall here is a climatology, not a gauge.** Open-Meteo is rate limited from the lab's IP, so this run used the documented Raipur figures. Check them against the nearest real gauge before costing a design.
- **A 1 m contour interval bounds everything downstream.** No depth in the output is better than about a metre, whatever the decimal places suggest.
- **Edge contact above 15% means the area is a lower bound.** The catchment carries on past the edge of the sheet. The recommended site here is at 3.0%, so this does not apply to it, but the field is in every response because on another sheet it will.
- **The ensemble does not run on this container.** See section 7.

10 AI usage

An AI coding assistant was used during development. It was useful for boilerplate, for first drafts of documentation, and for working through bugs on the deployed container.

Everything it suggested was reviewed and tested before it stayed in. The method, the siting rules, the validation approach and the deployment choices are my own. Every number in this report comes from a run against the real service, and the 451 tests exist so that no part of the pipeline is trusted just because it was written confidently.

11 Where each requirement is answered

REQUIREMENT	SECTION
GitHub repository	1. Links
Working API route URL	1. Links and 7. Deployment, verified in 6
Catchment estimation approach	3. How the catchment is estimated
Demonstration on the provided contour map	4. Demonstration
API documentation	5. API documentation, plus /docs and docs/API.md
Evaluation	6. Evaluation
Code extensibility to future phases	8. Extending to Phase 3